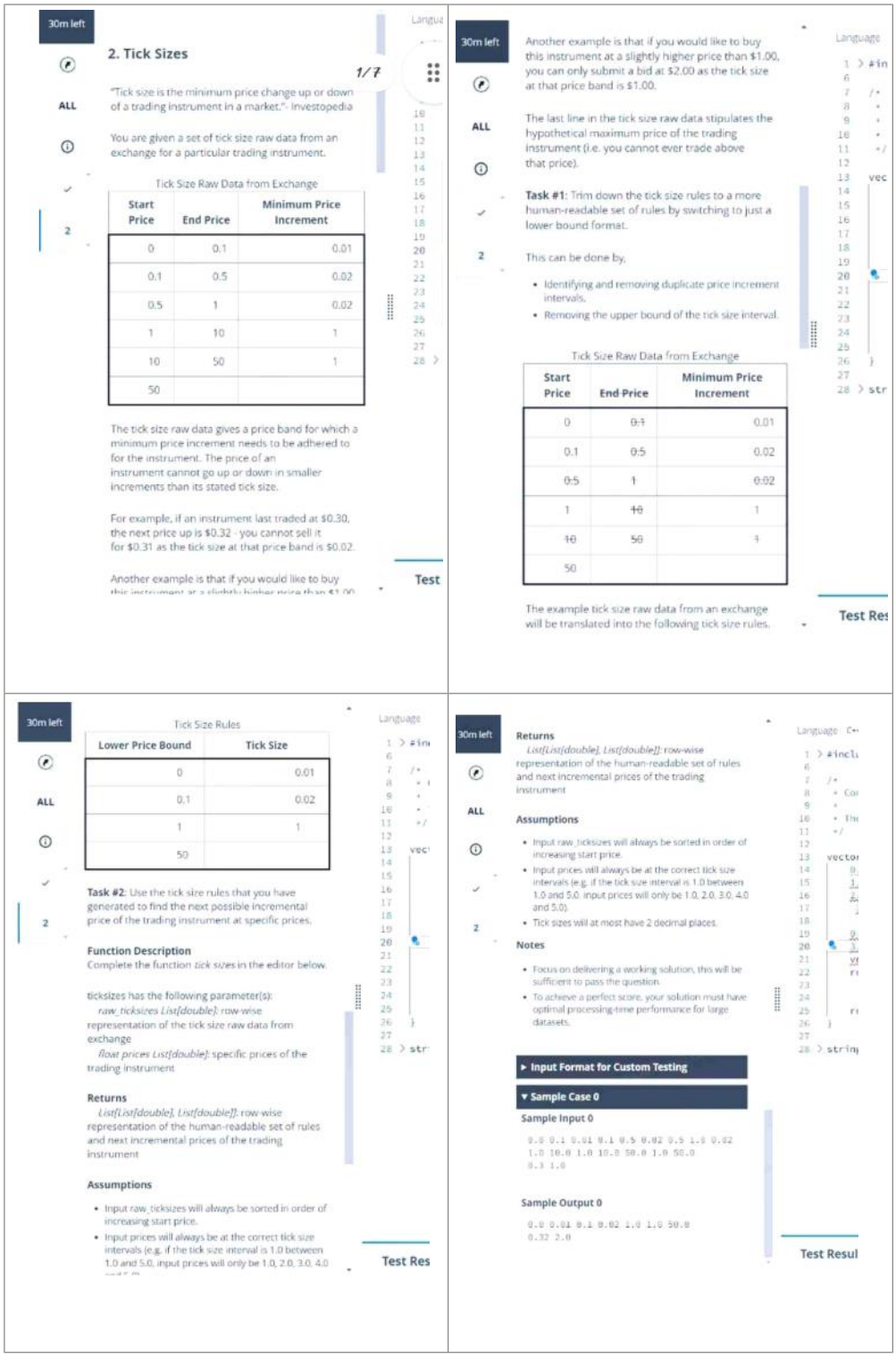
SDE

30 September 2024 11:33

QUESTION 1




```
#include <iostream>
#include <vector>
#include <algorithm>
#include <iomanip>
```

```
using namespace std;

struct TickSizeRule {
   double lowerPriceBound;
   double tickSize;
};

void trimTickSizeRules(const vector<vector<double>> &raw_tickSizes, vector<TickSizeRule>
&tickSizeRules) {
   double prevLower = raw_tickSizes[0][0];
   double prevTickSize = raw_tickSizes[0][2];

   tickSizeRules.push_back({prevLower, prevTickSize});

   for (int i = 1; i < raw_tickSizes.size(); ++i) {
      double currentLower = raw_tickSizes[i][0];
      double currentTickSize = raw_tickSizes[i][2];

      if (currentTickSize != prevTickSize) {
         tickSizeRules.push_back({currentLower, currentTickSize});
         prevTickSize = currentTickSize;
      }
   }
}

double findNextIncrement(double price, const vector<TickSizeRule> &tickSizeRules) {
   auto it = lower_bound(tickSizeRules.begin(), tickSizeRules.end(), price, [](const TickSizeRule &rule,
double value) {
      return rule.lowerPriceBound <= value;
   });

   if (it != tickSizeRules.begin()) {
      --it;
   }

   return price + it->tickSize;
}

int main() {
   vector<vector<double>> raw_tickSizes = {
       {0.0, 0.2, 0.1},

      {0.2, 0.6, 0.2},

      {0.6, 1.0, 0.2},
```

```
        {1.0, 1.1, 0.1},

        {1.1, 1.6, 0.5},

        {1.6,2.8,0.3},

        {2.8}
    };

    vector<double> prices = {0.5,1.3,2};

    vector<TickSizeRule> tickSizeRules;
    trimTickSizeRules(raw_tickSizes, tickSizeRules);

    for (const auto &rule : tickSizeRules) {
        cout << fixed << setprecision(2) << rule.lowerPriceBound << " " << rule.tickSize << endl;
    }

    for (double price : prices) {
        double nextPrice = findNextIncrement(price, tickSizeRules);
        cout << fixed << setprecision(2) << nextPrice << endl;
    }

    return 0;
}
```

QUESTION 2

https://leetcode.com/problems/search-suggestions-system/description/?envType=problem-list-v2&envId=trie

Input: vector of strings named words, string named search

Todo: Think of a search engine that has the words vector as its database. string search is your current keyword. You start typing the search word and your search engine starts giving 3 suggestions whose prefixes match the current state of your search word. So, as you write the search word character by character, you get a new recommendation list suited for that. If there are more than three words in the database that are suited, return the three lexicographically smallest ones. If there are none, return empty list.

Output: A vector<vector<string>> which contains the recommendations for all characters being typed one by one.

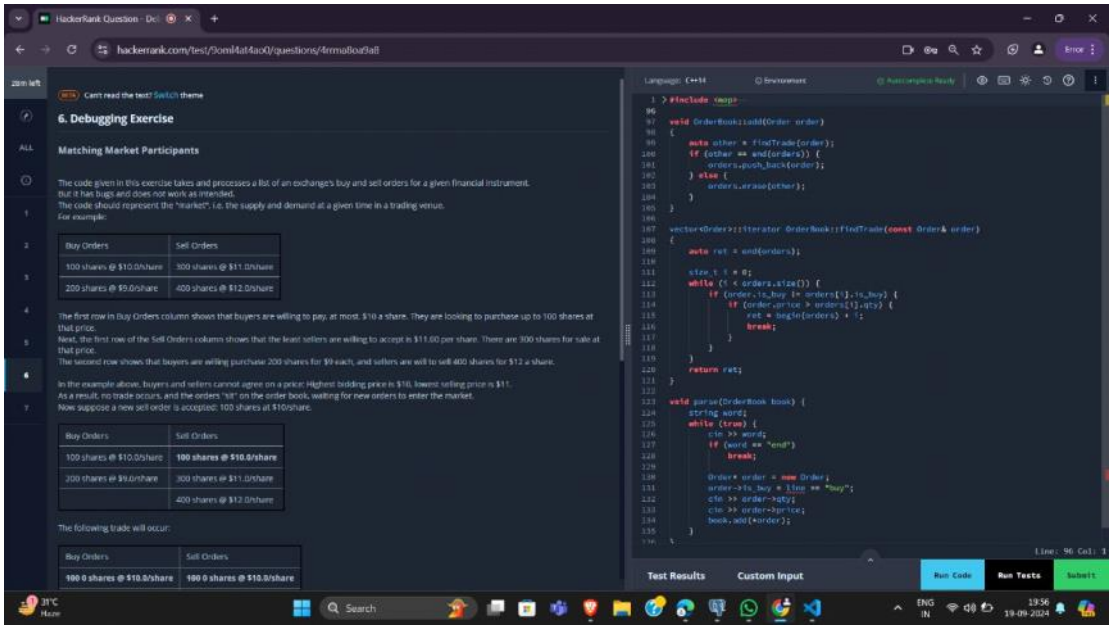
Example: words : ["cart", "carpet", "carpool", "call"], search: carl

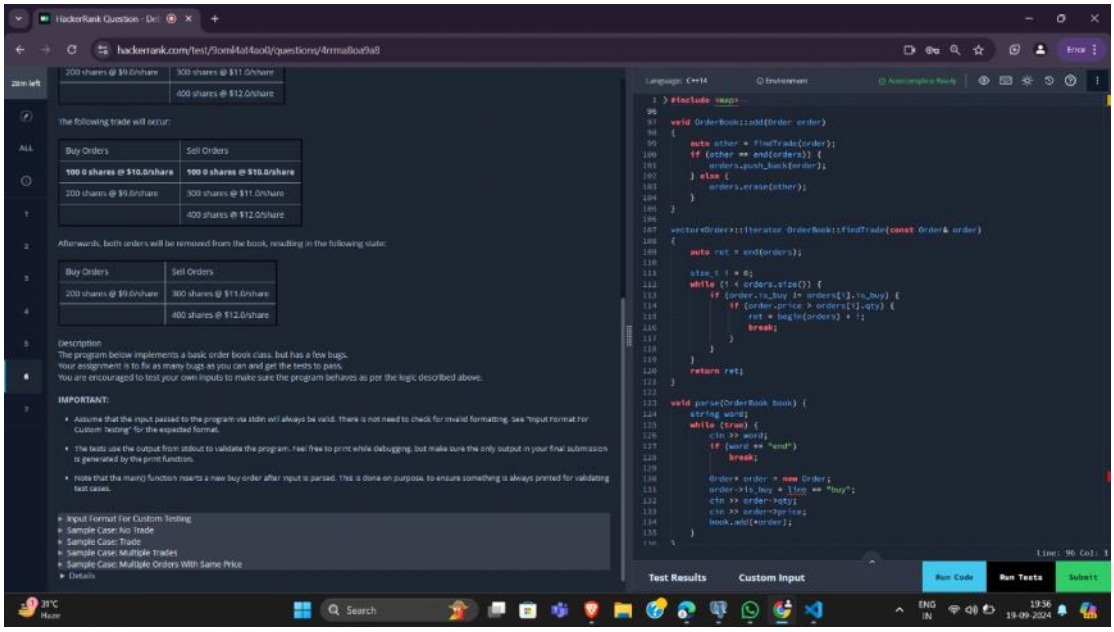
output:
[
c : ["call", "carpet", "carpool"],
ca : ["call", "carpet", "carpool"],
car : ["carpet", "carpool", "cart"],
carl : []

]

```
#include <bits/stdc++.h>
#define ll long long
class Solution {
public:
    vector<vector<string>> suggestedProducts(vector<string>& products, string
searchWord) {
        sort(products.begin(), products.end());
        vector<vector<string>>result;
        string currentPrefix = "";
        for(auto&c : searchWord){
            currentPrefix+=c; // Build the prefix incrementally
            vector<string> suggestions;
            //  Use lower_bound to find the first product that is not less than the
current prefix
            auto it = lower_bound(products.begin(), products.end(), currentPrefix);
            // Collect up to three valid suggestions
            int i = 0;
        while(i < 3 && it != products.end() && it->substr(0, currentPrefix.size())
== currentPrefix){
            suggestions.push_back(*it);
            ++i;
            ++it;
        }
            result.push_back(suggestions);
        }

        return result;
    }
};
```

QUESTION 3:



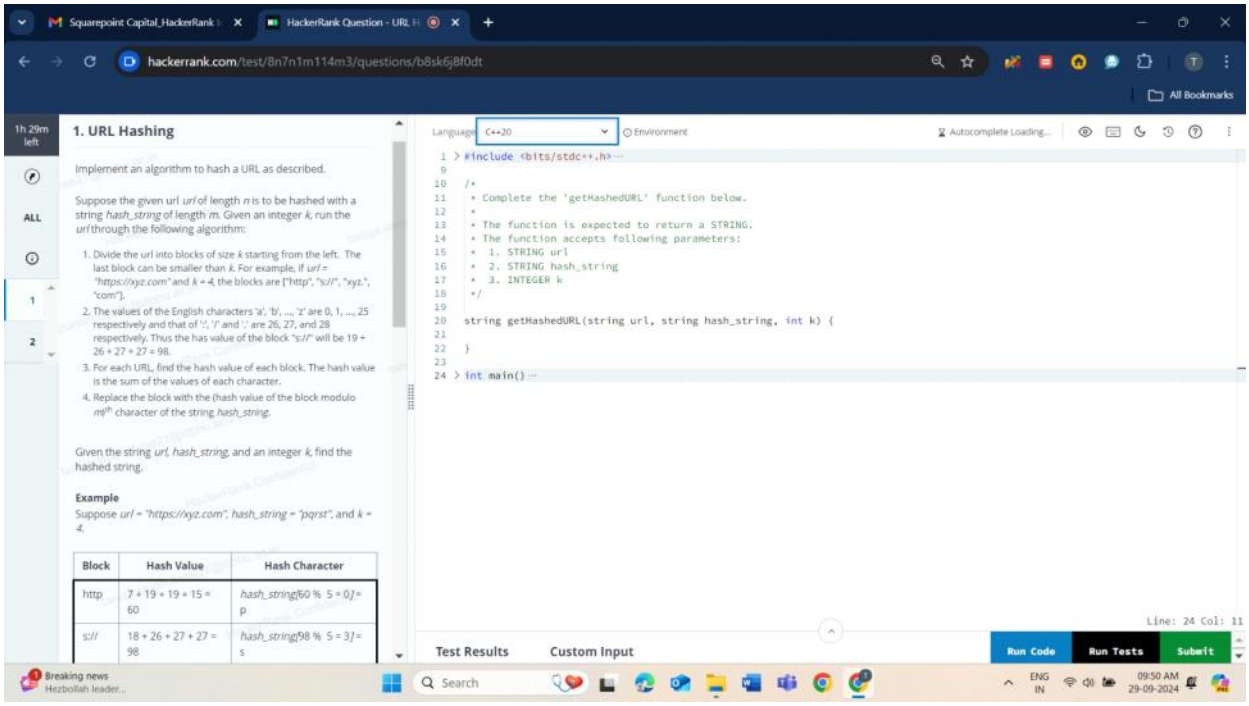


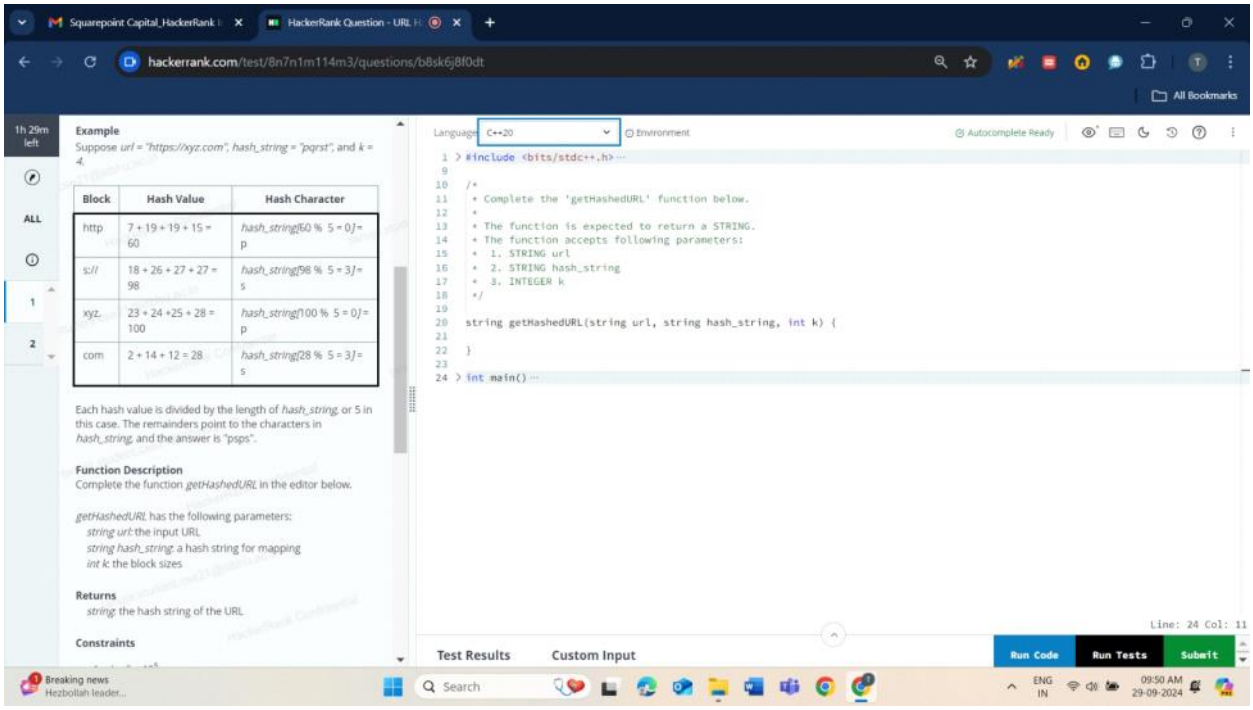
```
#include <map>
#include <set>
#include <list>
#include <cmath>
#include <ctime>
#include <deque>
#include <queue>
#include <stack>
#include <string>
#include <bitset>
#include <cstdio>
#include <limits>
#include <vector>
#include <climits>
#include <cstring>
#include <cstdlib>
#include <fstream>
#include <numeric>
#include <sstream>
#include <iomanip>
#include <iostream>
#include <algorithm>
#include <unordered_map>
using namespace std;
struct Order
{
    bool is_buy = false;
    int qty = 0;
    double price = 0.0;
};
ostream& operator <<(ostream& os, const Order& order)
{
    return os << (order.is_buy ? "buy " : "sell ")
        << order.qty << "@$" << setprecision(1)
        << fixed << order.price;
}
bool operator <(const Order& a, const Order& b)
{
     return a.price < b.price;
};
class OrderBook {
public:
    OrderBook() = default;

    // formats and prints the orders as the test cases expect.
```

```
    ~OrderBook() {
        auto itr = splitIntoBuyAndSellOrders();
        std::sort(begin(orders), itr);
        std::sort(itr, end(orders));
        print();
    }
    // checks the opposing side's available orders.
    // for a buy order, look at existing sell orders, and vice versa.
    // if a trade is possible, update the order book accordingly.
    // otherwise, insert the order into the book.
    void add(Order order);

    // print the order book
    // IMPORTANT: the test cases depend on this method.
    // please make sure this is the ONLY ouput that gets printed to stdout.
    void print();
private:
    // returns an iterator to the best "match" for a given order.
    // for buy orders, this would be the lowest sell price.
    // for sell orders, the highest buy price.
    // if no orders meet the criteria, return orders.end()
    vector<Order>::iterator findTrade(const Order& order);

    // split orders into buy and sell orders.
    // return an iterator to the first sell order.
    // all orders before the iterator are buy orders.
    // all orders after are sell orders.
    vector<Order>::iterator splitIntoBuyAndSellOrders();

    vector<Order> orders;
};
vector<Order>::iterator OrderBook::splitIntoBuyAndSellOrders() {
        return stable_partition(begin(orders),
                                end(orders),
                                [](const Order& order) { return order.is_buy; });
    }
void OrderBook::print() {
    for (const auto& order : orders)
        cout << order << endl;
}

void OrderBook::add(Order order) {
    while (order.qty > 0) {
        auto other = findTrade(order);
        if (other == end(orders)) {
            // Aggregate orders if there is an existing order with the same price and type
            auto it = std::find_if(orders.begin(), orders.end(), [&order](const Order&
existingOrder) {
                return existingOrder.is_buy == order.is_buy && existingOrder.price ==
order.price;
            });
            if (it != orders.end()) {
                it->qty += order.qty;
            } else {
                orders.push_back(order);
            }
            return;
        } else {
            // Match found, fulfill as much as possible
            if (order.qty >= other->qty) {
                order.qty -= other->qty;
                orders.erase(other);
            } else {
                other->qty -= order.qty;
                order.qty = 0;
            }
```

```
        }
    }
}
vector<Order>::iterator OrderBook::findTrade(const Order& order) {
    vector<Order>::iterator best_match = orders.end();
    for (auto it = orders.begin(); it != orders.end(); ++it) {
        if (order.is_buy != it->is_buy) {
            if (order.is_buy) {
                // Buy order: match with the lowest sell price
                if (it->price <= order.price) {
                    if (best_match == orders.end() || it->price < best_match->price) {
                        best_match = it;
                    }
                }
            } else {
                // Sell order: match with the highest buy price
                if (it->price >= order.price) {
                    if (best_match == orders.end() || it->price > best_match->price) {
                        best_match = it;
                    }
                }
            }
        }
    }
    return best_match;
}
void parse(OrderBook &book) {
    string word;
    while (true) {
        cin >> word;
        if (word == "end")
            break;
        Order order;
        order.is_buy = (word == "buy");
        cin >> order.qty >> order.price;
        book.add(order);
    }
}
int main() {
    OrderBook book;
    parse(book);
    Order order = {true, 10, 11.0};
    book.add(order);
}
```

URL HASHING

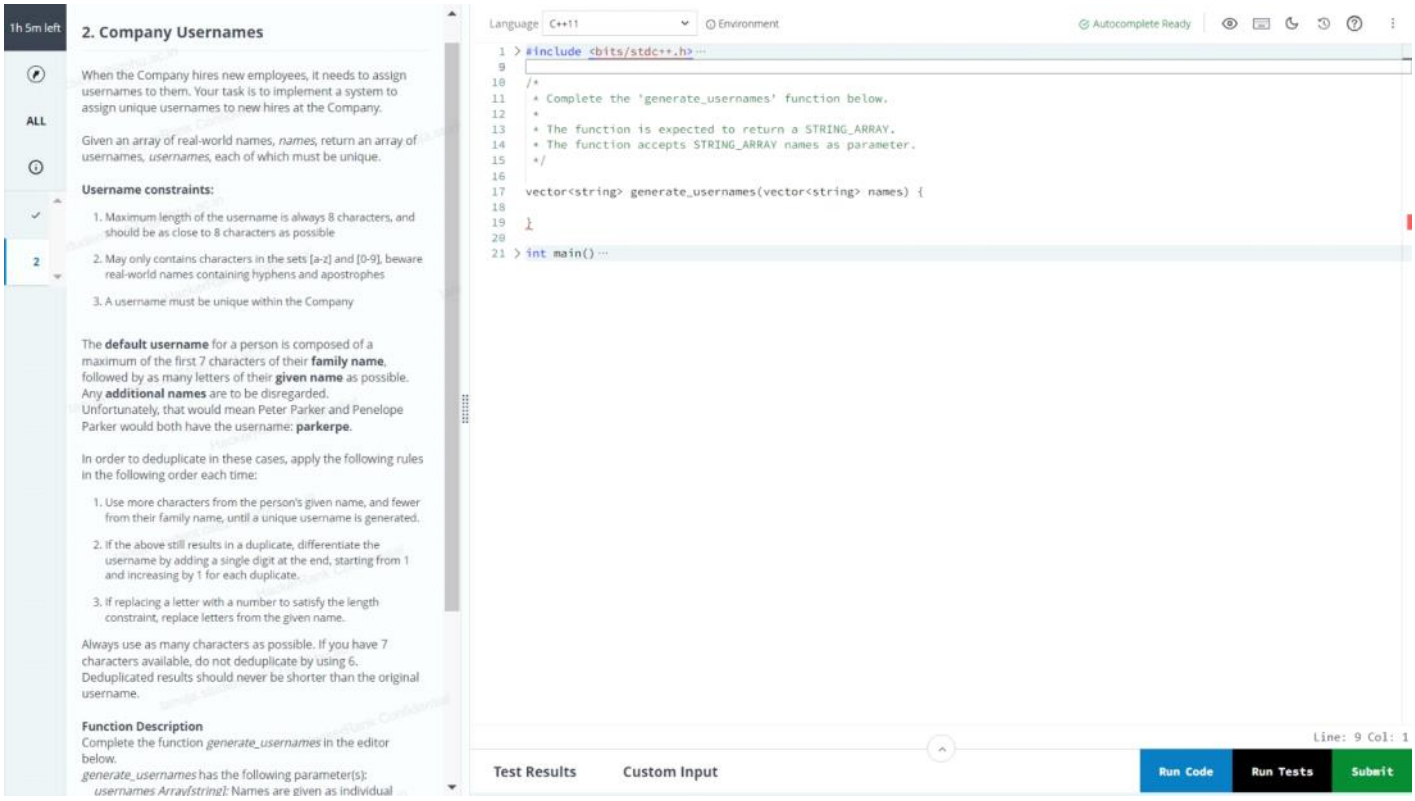


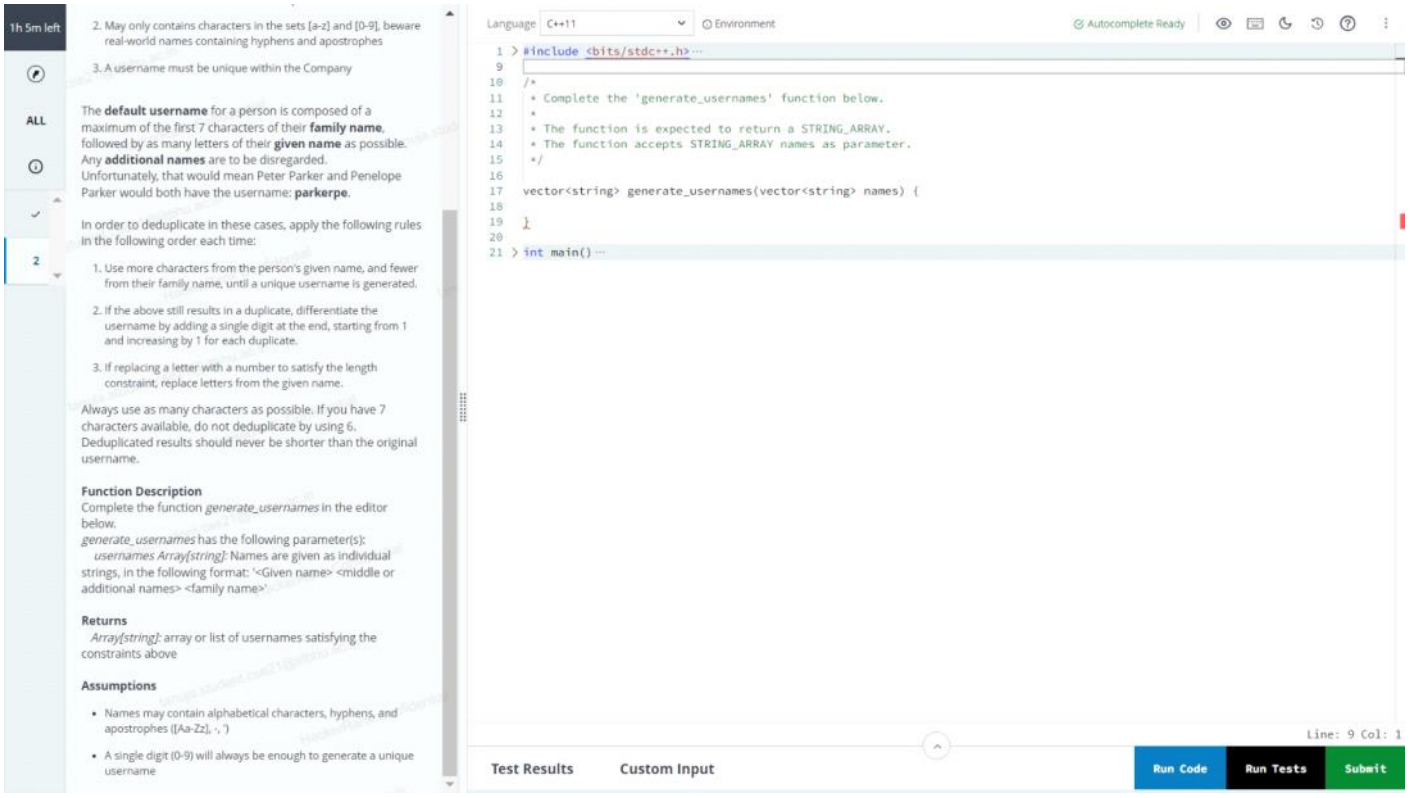



```
#include <iostream>
#include <string>
#include <vector>

using namespace std;

int charValue(char ch) {
    if (ch >= 'a' && ch <= 'z')
        return ch - 'a';
    if (ch == '/')
        return 26;
    if (ch == '.')
```

```
        return 27;
    return -1;
}

string hashURL(const string& url, const string& hash_string, int k) {
    int n = url.size();
    int m = hash_string.size();
    string result = "";

    for (int i = 0; i < n; i += k) {
        int hash_value = 0;

        for (int j = i; j < i + k && j < n; j++) {
            hash_value += charValue(url[j]);
        }

        result += hash_string[hash_value % m];
    }

    return result;
}

int main() {
    string url = "https://xyz.com";
    string hash_string = "pqrst";
    int k = 4;

    string hashed_url = hashURL(url, hash_string, k);

    cout << "Hashed URL: " << hashed_url << endl;

    return 0;
}
```

COMPANY USERNAMES:






```
#include <bits/stdc++.h>
using namespace std;

// Helper function to clean the input name (remove hyphens and apostrophes)
string clean_name(const string &name) {
    string result;
    for (char ch : name) {
        if (isalpha(ch)) {
            result += tolower(ch);
        }
```

```
    }
    return result;
}

// Function to generate unique usernames
vector<string> generate_usernames(vector<string> names) {
    unordered_map<string, int> username_count;  // To keep track of usernames used and their
counts
    vector<string> result;

    for (string &full_name : names) {
        // Split the given full name into parts (given name, middle name(s), family name)
        vector<string> name_parts;
        stringstream ss(full_name);
        string word;

        while (ss >> word) {
            name_parts.push_back(clean_name(word));
        }

        if (name_parts.size() < 2) continue; // Skip if there is not enough data to generate username

        string given_name = name_parts[0];
        string family_name = name_parts.back();

        // Start with a default username: first 7 characters of family name + as much of given name as
possible
        string username = family_name.substr(0, min(7, (int)family_name.size()));
        int remaining_length = 8 - username.length();
        if (remaining_length > 0) {
            username += given_name.substr(0, remaining_length);
        }

        // Attempt to adjust the username to make it unique by changing family/given name
composition
        int family_chars = min(7, (int)family_name.size());
        int given_chars = remaining_length;

        while (username_count.find(username) != username_count.end()) {
            if (family_chars > 1) {
                // Reduce the number of characters taken from the family name, and increase from given
name
                family_chars--;
                given_chars = 8 - family_chars;
                username = family_name.substr(0, family_chars) + given_name.substr(0, given_chars);
            } else {
                // If reducing family name characters is not enough, append a number to make it unique
                int count = ++username_count[username];
                if (username.length() == 8) {
                    // Replace the last character with a number
                    username[7] = '0' + count;
                } else {
                    // Append a number to the username
                    username += to_string(count);
                }
            }
        }

        // Store the final unique username
        username_count[username]++;
        result.push_back(username);
    }
```

```
    return result;
}

int main() {
    vector<string> names = {
        "Peter Parker",
        "Penelope Parker",
        "Tony Stark",
        "Bruce Banner",
        "Steve Rogers",
        "Natasha Romanoff",
        "Peter Parker",  // Duplicate test
        "Penelope Parker"  // Another duplicate test
    };

    vector<string> usernames = generate_usernames(names);

    for (const string &username : usernames) {
        cout << username << endl;
    }

    return 0;
}
```

QUESTION 7

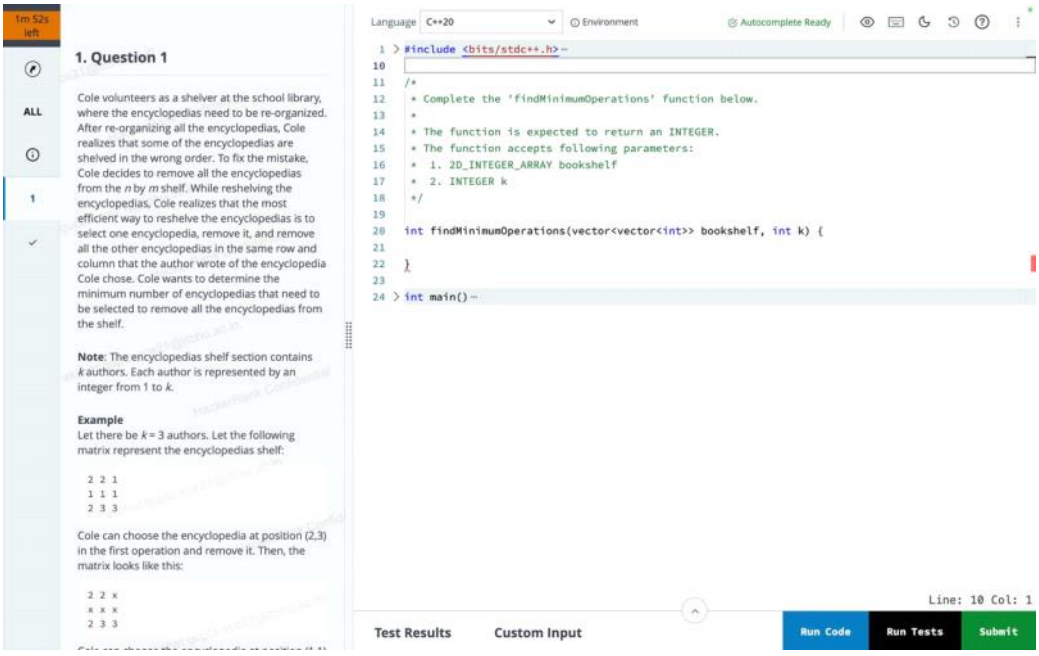
https://leetcode.com/problems/subarray-product-less-than-k/description/

```
class Solution {
public:
    int numSubarrayProductLessThanK(vector<int>& nums, int k) {
        if (k <= 1) {
            return 0;
        }

        int i = 0, j = 0;
        int n = nums.size();
        int product = nums[0];
        int ans = 0;

        nums.push_back(1);

        while (j < n) {
            if (product < k) {
                product *= nums[++j];
            } else {
                ans += j - i;
                product /= nums[i++];
            }
```

```
        }

        ans += (j - i) * (j - i + 1) / 2;

        return ans;
    }
};
```

DQA

30 September 2024 11:53



Soln

```
#include <bits/stdc++.h>
using namespace std;

class DisjointSet {
   vector<int> rank, parent, size;
public:
   DisjointSet(int n) {
      rank.resize(n + 1, 0);
      parent.resize(n + 1);
      size.resize(n + 1);
      for (int i = 0; i <= n; i++) {
         parent[i] = i;
         size[i] = 1;
      }
   }

   int findUPar(int node) {
      if (node == parent[node])
         return node;
      return parent[node] = findUPar(parent[node]);
   }
```

```
    void unionBySize(int u, int v) {
        int ulp_u = findUPar(u);
        int ulp_v = findUPar(v);
        if (ulp_u == ulp_v) return;
        if (size[ulp_u] < size[ulp_v]) {
            parent[ulp_u] = ulp_v;
            size[ulp_v] += size[ulp_u];
        }
        else {
            parent[ulp_v] = ulp_u;
            size[ulp_u] += size[ulp_v];
        }
    }
};

class Solution {
public:
    int minRemovals(vector<vector<int>>& shelf) {
        int n = shelf.size();
        int m = shelf[0].size();

        // MaxRow and MaxCol are based on matrix dimensions, we treat rows and columns separately
        DisjointSet ds(n * m);
        unordered_map<int, int> componentNodes;

        // Loop through the grid and unite nodes based on the same row and column condition
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                // Row-wise union if the same value appears in the row
                for (int k = j + 1; k < m; k++) {
                    if (shelf[i][j] == shelf[i][k]) {
                        ds.unionBySize(i * m + j, i * m + k); // Connect row cells if values are the same
                    }
                }

                // Column-wise union if the same value appears in the column
                for (int k = i + 1; k < n; k++) {
                    if (shelf[i][j] == shelf[k][j]) {
                        ds.unionBySize(i * m + j, k * m + j); // Connect column cells if values are the same
                    }
                }

                // Mark the current cell as seen
                componentNodes[i * m + j] = 1;
            }
```

```
        }

        // Count unique components
        int cnt = 0;
        for (auto it : componentNodes) {
            if (ds.findUPar(it.first) == it.first) {
                cnt++;
            }
        }

        // The number of removals is the total number of components
        return cnt;
    }
};

int main() {
    vector<vector<int>> shelf = {
        {2, 2, 1},
        {2, 1, 1},
        {3, 3, 3}
    };

    Solution sol;
    cout << "Minimum removals to clean the shelf: " << sol.minRemovals(shelf) << endl;

    return 0;
}
```

```
1 1 2
2 2 2
1 2 2

output = 3
```

Alt Ans.

```
def find_idx(L):
    num=max(L)
    for j in range (0,len(L)):
        if L[j]==num:
            return j

def get(L):
    ans=0
    visited=[False]*len(L)
```

```
    while False in visited:
        x=[0]*n
        y=[0]*m
        for j in range (0,len(L)):
            if visited[j]==True:
                continue
            u,v=L[j]
            x[u]=x[u]+1
            y[v]=y[v]+1
        u=find_idx(x)
        v=find_idx(y)
        for j in range (0,len(L)):
            if L[j][0]==u or L[j][1]==v:
                visited[j]=True
        ans=ans+1
    return ans




for j in range (0,len(L)):
    for k in range (0,len(L)):
        M=dic.get(L[j][k],[])
        M.append([j,k])
        dic[L[j][k]]=M
ans=0
for x in dic:
    count=get(dic[x])
    ans=ans+count
print(ans)
```

C++

```
#include <iostream>
#include <vector>
#include <unordered_map>
#include <algorithm>

using namespace std;

int find_idx(const vector<int> &L) {
    int num = *max_element(L.begin(), L.end());
    for (int j = 0; j < L.size(); ++j) {
        if (L[j] == num) {
            return j;
```

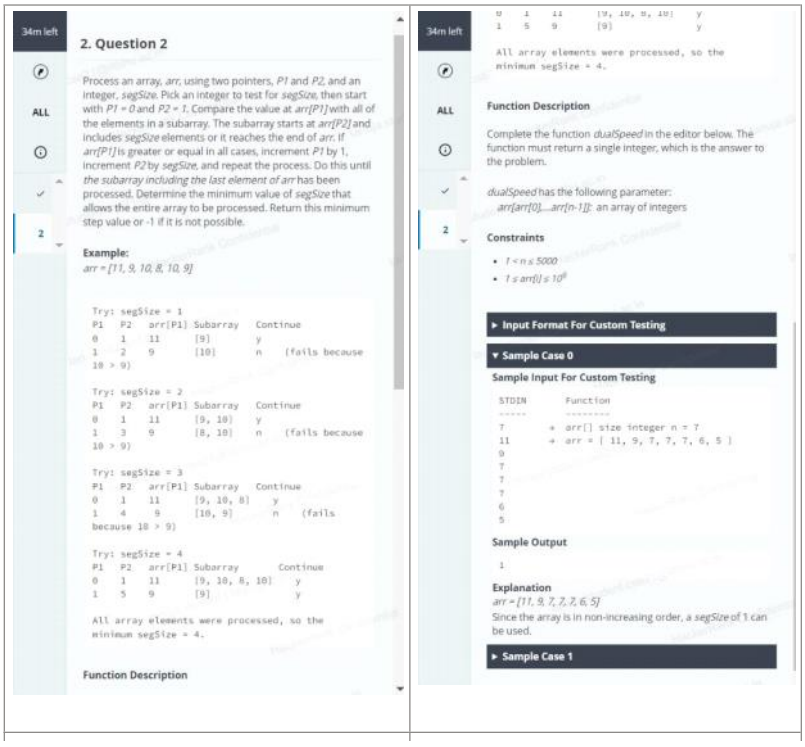
```
        }
    }
    return -1;  // This line should never be reached if the list is non-empty.
}

int get(const vector<pair<int, int>> &L, int n, int m) {
    int ans = 0;
    vector<bool> visited(L.size(), false);

    while (find(visited.begin(), visited.end(), false) != visited.end()) {
        vector<int> x(n, 0), y(m, 0);

        for (int j = 0; j < L.size(); ++j) {
            if (visited[j]) {
                continue;
            }

            int u = L[j].first;
            int v = L[j].second;
            x[u]++;
            y[v]++;
        }

        int u = find_idx(x);
        int v = find_idx(y);

        for (int j = 0; j < L.size(); ++j) {
            if (L[j].first == u || L[j].second == v) {
                visited[j] = true;
            }
        }

        ans++;
    }

    return ans;
}

int main() {
    int n = 3, m = 3, k = 2;
    vector<vector<int>> L = {
        {1, 1, 2},
        {2, 2, 2},
        {1, 2, 2}
    };
```

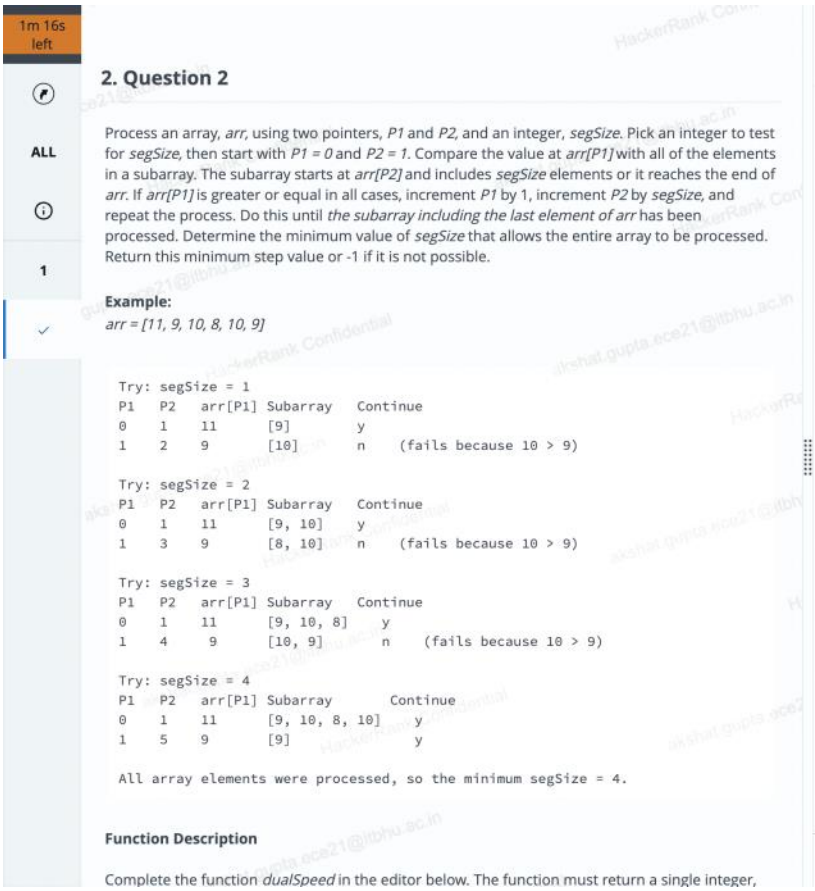
```
    unordered_map<int, vector<pair<int, int>>> dic;

    // Create the dictionary to group indices by value
    for (int j = 0; j < L.size(); ++j) {
        for (int k = 0; k < L[j].size(); ++k) {
            dic[L[j][k]].emplace_back(j, k);
        }
    }

    // Calculate the answer
    int ans = 0;
    for (auto &entry : dic) {
        int count = get(entry.second, n, m);
        ans += count;
    }

    cout << ans << endl;

    return 0;
}
```

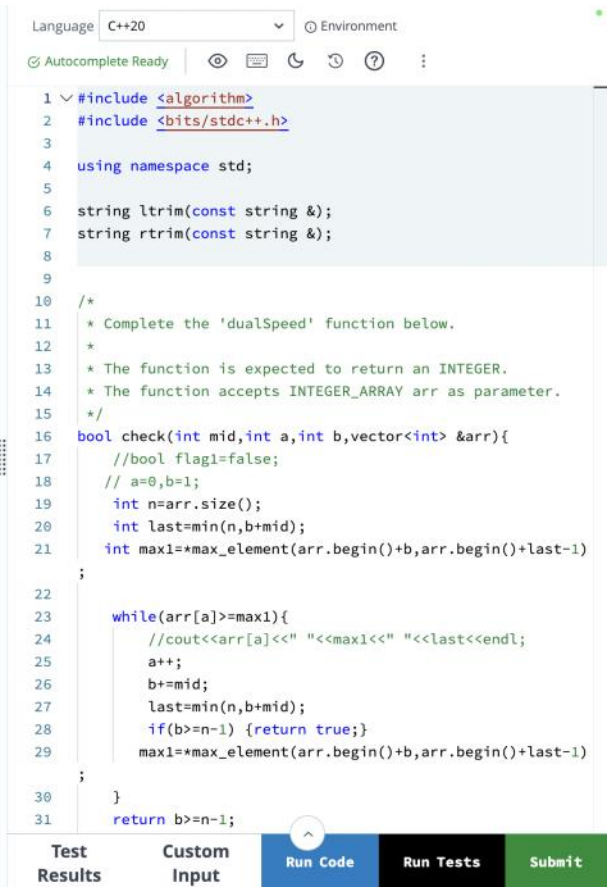
Hackerank Approach

For each unique value from 1 to K, greedily find the best intersection point, destroy the cells, keep doing the same until all cells with that value has been destroyed. To find the best intersection point it takes O(n*m), and the number of steps to destroy all would be at most (n,m) for each unique value, and there are at most K unique values. Therefore, the overall time complexity would be O(k*min(n,m)*n*m). Considering the worst case where n=m=100, k=50, it turns out to be O(50*100*100*100) = O(5*10^7).

QUESTION 2:







started with segsize = 1 , checked condition at each step untill the subarray having last element

if condition met return segsize , else continue to segsize + 1 and repeat the process.

```
#include<bits/stdc++.h>
bool canProcessWithSegSize(const std::vector<int>& arr, int segSize) {
   int n = arr.size();
   int P1 = 0;
   int P2 = 1;
   while (P1 < n && P2 < n)
   {
      bool isValid = true;
      for (int i = 0; i < segSize && P2 + i < n; ++i)
      {
         if (arr[P1] < arr[P2 + i])
         {
            isValid = false;
            break;
         }
      }

      if (isValid)
```

```
        {
            P1++;
            P2 += segSize;
        } else
        {
            return false;
        }
    }

    return true;
}

int findMinimumSegSize(const std::vector<int>& arr) {
    int n = arr.size();

    if (n <= 1) {
        return 1;
    }

    for (int segSize = 1; segSize < n; ++segSize) {
        if (canProcessWithSegSize(arr, segSize)) {
            return segSize;
        }
    }

    return -1;
}

int main() {
    std::vector<int> arr = {11,9,10,8,10,9};  // Example array

    int result = findMinimumSegSize(arr);

    if (result != -1) {
        std::cout << "Minimum segSize: " << result << std::endl;
    } else {
        std::cout << "No valid segSize found." << std::endl;
    }

    return 0;
}
```

INFRA

30 September 2024 11:53

LINUX

7. Linux: Users Bulk Onboarding
Complete the file stub "/home/ubuntu/900026- linux-users-bulk-onboarding/script.sh" with one or more steps that do the following:
• Read the list of first and last names of employers to onboard from "/home/ubuntu/900026-linux-users-bulk-onboarding/onboard.txt".
• Create the users with logins in the format "f_lastname", where "f" is the lowercase first letter of the first name.
• Put all the created users in the "onboarding" group.
• Force all created users to update their passwords at the next login.

Note
The completed solution will be evaluated in a new, clean environment. Be sure everything is in the "/home/ubuntu/900026-linux-users-bulk- onboarding" folder.
All the tasks should be done within a simple "sudo solve" execution invoked from the question directory.
• You have sudo access.

Grading
• The execution result of "sudo solve" invoked from the question directory solves the task.

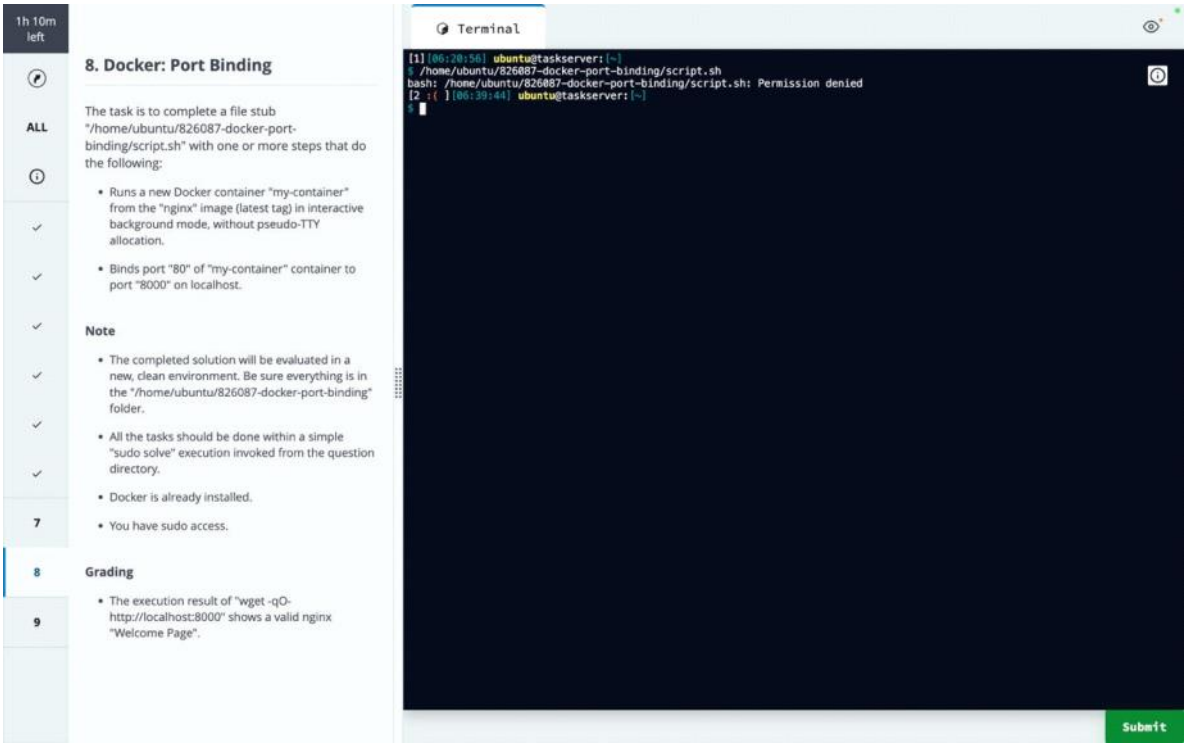
Solution

```
#!/bin/bash

# Define the file path
FILE="/home/ubuntu/900026-linux-users-bulk-onboarding/onboard.txt"

# Check if the file exists
if [ ! -f "$FILE" ]; then
    echo "File not found!"
    exit 1
fi

# Create the "onboarding" group if it doesn't exist
if ! getent group onboarding >/dev/null; then
    sudo groupadd onboarding
fi

# Loop through the onboard.txt file
while IFS=',' read -r firstname lastname
do
    # Format the username as f_lastname (lowercase first letter of the first name)
    username="${firstname:0:1,,}_$lastname"

    # Create the user with the formatted username
    if id "$username" &>/dev/null; then
        echo "User $username already exists, skipping..."
    else
        sudo useradd -m -g onboarding "$username"
        echo "User $username created."
    fi

    # Force user to change password at next login
    sudo passwd -e "$username"

done < "$FILE"

echo "All users onboarded successfully."
```

Running the command

```
sudo chmod +x /home/ubuntu/900026-linux-users-bulk-onboarding/script.sh

sudo /home/ubuntu/900026-linux-users-bulk-onboarding/script.sh
```

DOCKER:



Step 1: Create the script directory

First, ensure that the target directory exists. If it does not, create it:

bash
Copy code
mkdir -p /home/ubuntu/826087-docker-port-binding

Step 2: Create or edit the script file

You can use a text editor like nano to create the script:

bash
Copy code
nano /home/ubuntu/826087-docker-port-binding/script.sh

Step 3: Add the necessary commands to the script

In the nano editor, add the following lines to the script:

bash
Copy code
#!/bin/bash
# Run a new Docker container from the nginx image
sudo docker run -d --name my-container -p 8000:80 nginx

Step 4: Save and exit the editor

- Press CTRL + X to exit.
- Press Y to confirm saving changes.
- Press ENTER to save with the same filename.

Step 5: Ensure the script has execute permissions

Run the following command to make the script executable:

bash
Copy code
chmod +x /home/ubuntu/826087-docker-port-binding/script.sh

Step 6: Run the script

Change to the script's directory and execute it:

bash
Copy code
cd /home/ubuntu/826087-docker-port-binding
sudo ./script.sh

Step 7: Verify that the container is running

You can check that the Nginx welcome page is accessible by executing:

bash
Copy code
wget -qO- http://localhost:8000

Expected Output

If everything is set up correctly, you should see the HTML content of the Nginx welcome page as the output of the wget command.

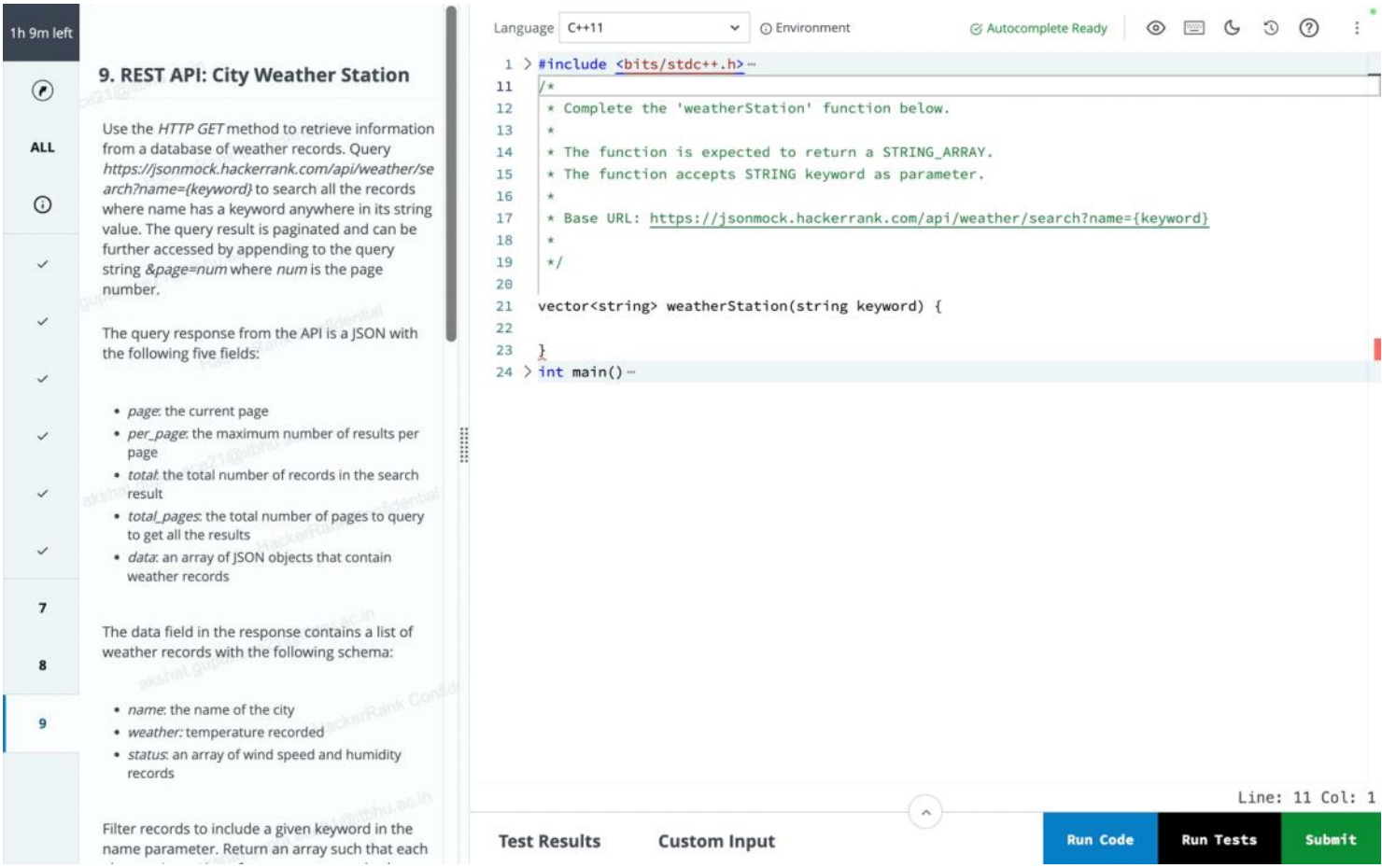
Summary of Commands

Here's a summary of all the commands you need to run in order:

bash
Copy code
mkdir -p /home/ubuntu/826087-docker-port-binding
nano /home/ubuntu/826087-docker-port-binding/script.sh
(Add the script content and save)

chmod +x /home/ubuntu/826087-docker-port-binding/script.sh
cd /home/ubuntu/826087-docker-port-binding
sudo ./script.sh
wget -qO- http://localhost:8000
This will create the required Docker container and bind the specified ports.

REST API



1h 9m left

ALL

Filter records to include a given keyword in the name parameter. Return an array such that each element is a string of comma-separated values: *city_name, temperature, wind, humidity.*

For example,

```
{
  "name": "Adelaide",
  "weather": "15 degree",
  "status": [
    "Wind: 8Kmph",
    "Humidity: 61%"
  ]
}
```

The JSON record is stored as *Adelaide,15,8,61*
Return the list sorted by city name.

Function Description
Complete the function *weatherStation(keyword)* in the editor below.

weatherStation has the following parameter(s):
string keyword: the string to search

Returns
string[]: the weather data for each city

Note: Please review the header in the code stub to see available libraries for API requests in the selected language. Required libraries can be imported in order to solve the question. Check our full list of supported libraries at https://www.hackerrank.com/environment.

▸ **Input Format For Custom Testing**

▾ **Sample Case 0**

Sample Input For Custom Testing

Language C++11 | Environment | Autocomplete Ready

```
#include <bits/stdc++.h>
/*
 * Complete the 'weatherStation' function below.
 *
 * The function is expected to return a STRING_ARRAY.
 * The function accepts STRING keyword as parameter.
 *
 * Base URL: https://jsonmock.hackerrank.com/api/weather/search?name={keyword}
 *
 */

vector<string> weatherStation(string keyword) {

}
int main()
```

Line: 11 Col: 1

Test Results **Custom Input** Run Code | Run Tests | Submit

1h 9m left

ALL

Returns
string[]: the weather data for each city

Note: Please review the header in the code stub to see available libraries for API requests in the selected language. Required libraries can be imported in order to solve the question. Check our full list of supported libraries at https://www.hackerrank.com/environment.

▸ **Input Format For Custom Testing**

▾ **Sample Case 0**

Sample Input For Custom Testing

```
all
```

Sample Output

```
Dallas,12,2,5
Dallupura,10,9,14
Vallejo,1,24,56
```

Explanation

The list of cities that contain the keyword 'all' and their data is shown as comma-separated values.

▾ **Sample Case 1**

Sample Input For Custom Testing

```
io
```

Sample Output

```
Qiongshan,3,31,47
```

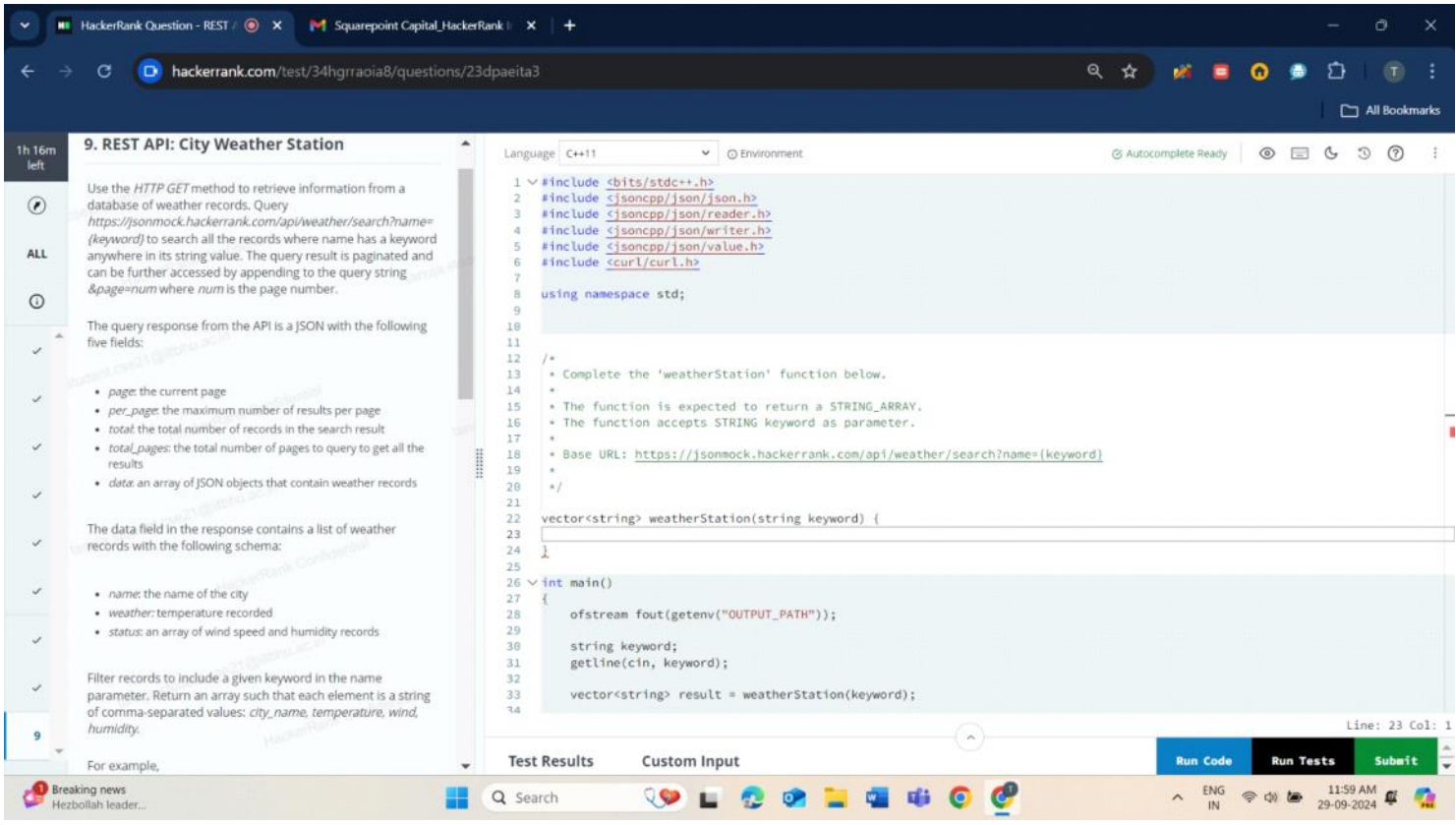
Explanation
Only one city contains the keyword 'io'.

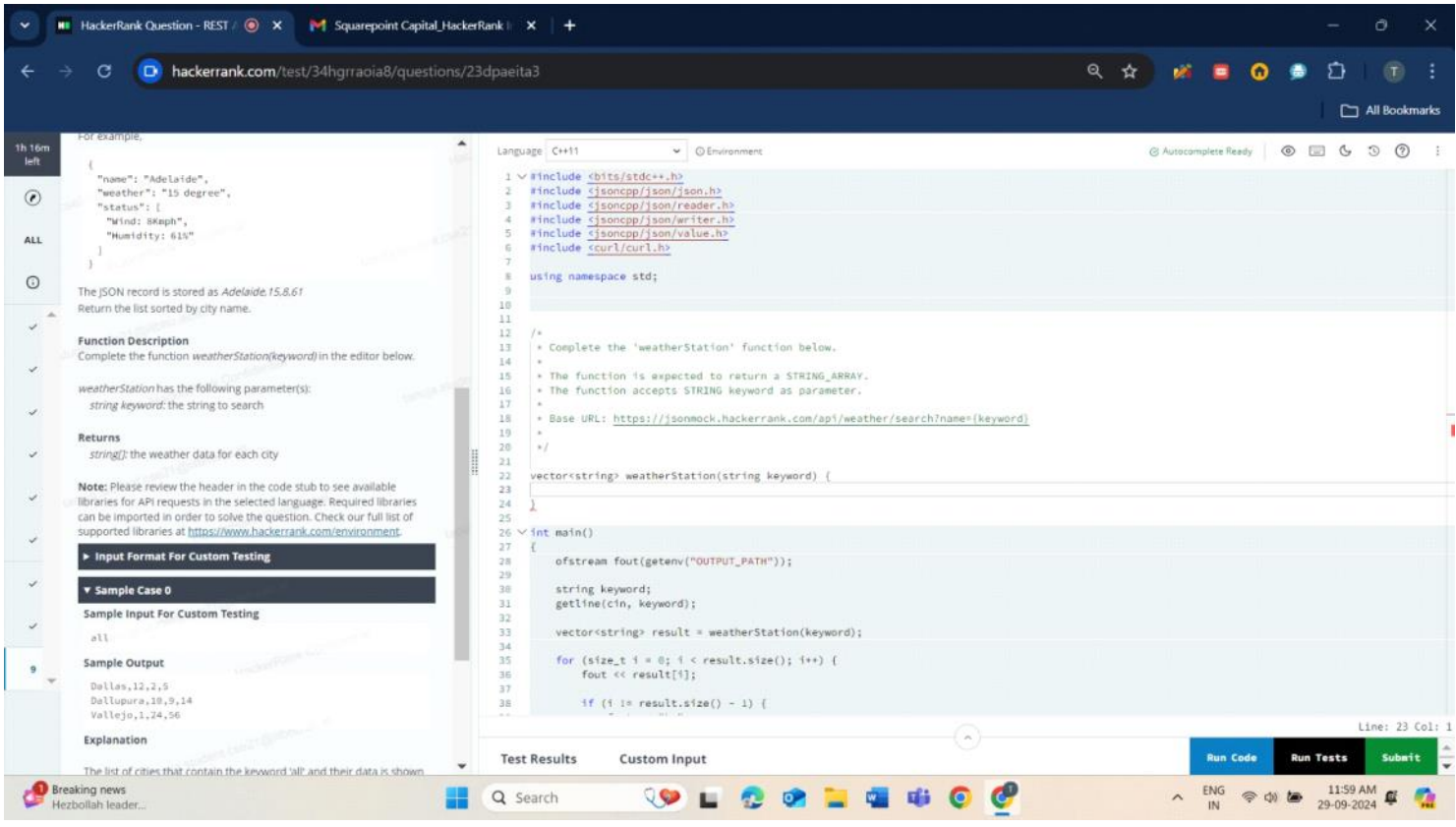
Language C++11 | Environment | Autocomplete Ready

```
#include <bits/stdc++.h>
/*
 * Complete the 'weatherStation' function below.
 *
 * The function is expected to return a STRING_ARRAY.
 * The function accepts STRING keyword as parameter.
 *
 * Base URL: https://jsonmock.hackerrank.com/api/weather/search?name={keyword}
 *
 */

vector<string> weatherStation(string keyword) {

}
int main()
```

Line: 11 Col: 1

Test Results **Custom Input** Run Code | Run Tests | Submit






```
#include <iostream>
#include <string>
#include <vector>
#include <sstream>
#include <algorithm>
#include <curl/curl.h>
#include <json/json.h> // Make sure you have a JSON library installed

using namespace std;

// Callback function to handle the response data from cURL
size_t WriteCallback(void* contents, size_t size, size_t nmemb, string* userp) {
    size_t totalSize = size * nmemb;
    userp->append((char*)contents, totalSize);
    return totalSize;
```

```
}

vector<string> weatherStation(string keyword) {
    vector<string> results;
    CURL* curl;
    CURLcode res;
    string readBuffer;

    // Initialize cURL
    curl_global_init(CURL_GLOBAL_DEFAULT);
    curl = curl_easy_init();

    if (curl) {
        // Set the URL for the API request
        string url = "https://jsonmock.hackerrank.com/api/weather/search?name=" + keyword;
        curl_easy_setopt(curl, CURLOPT_URL, url.c_str());
        curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, WriteCallback);
        curl_easy_setopt(curl, CURLOPT_WRITEDATA, &readBuffer);

        // Perform the request
        res = curl_easy_perform(curl);

        // Check for errors
        if (res != CURLE_OK) {
            cerr << "curl_easy_perform() failed: " << curl_easy_strerror(res) << endl;
            return results;
        }

        // Clean up
        curl_easy_cleanup(curl);
    }

    curl_global_cleanup();

    // Parse the JSON response
    Json::Value jsonData;
    Json::Reader jsonReader;

    if (jsonReader.parse(readBuffer, jsonData)) {
        // Process the 'data' array from the JSON response
        const Json::Value data = jsonData["data"];

        // Iterate through each record
        for (const auto& record : data) {
            string cityName = record["name"].asString();
            string temperature = record["weather"].asString();
            const Json::Value status = record["status"];

            // Assuming status contains "Wind: XKmph" and "Humidity: Y%"
            string wind = status[0].asString(); // Get wind status
            string humidity = status[1].asString(); // Get humidity status

            // Extract numerical values for wind and humidity
            size_t windStart = wind.find(':') + 2; // Find wind speed after "Wind: "
            size_t windEnd = wind.find('K', windStart); // Find 'K'
            string windSpeed = wind.substr(windStart, windEnd - windStart); // Extract wind speed

            size_t humidityStart = humidity.find(':') + 2; // Find humidity after "Humidity: "
            size_t humidityEnd = humidity.find('%', humidityStart); // Find '%'
            string humidityValue = humidity.substr(humidityStart, humidityEnd - humidityStart); // Extract
humidity

            // Create a result string in the desired format
            results.push_back(cityName + ", " + temperature + ", " + windSpeed + ", " + humidityValue);
        }
    } else {
        cerr << "Failed to parse JSON data" << endl;
    }

    // Sort results by city name
    sort(results.begin(), results.end());

    return results;
}

int main() {
    string keyword;
    cout << "Enter the keyword: ";
    cin >> keyword;

    vector<string> weatherData = weatherStation(keyword);
    for (const auto& record : weatherData) {
        cout << record << endl;
    }

    return 0;
}
```

MCQ

30 September 2024 12:06